What a program is: your first Python

Programs as exact steps, a spreadsheet as a program, and your first lines of Python with their real output.

THE QUESTION

What is a program, and what happens when Python runs the one you wrote?

THE INTUITION

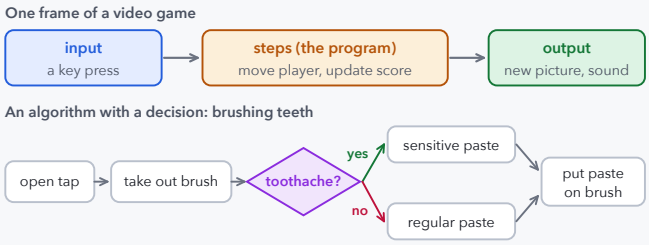
A program is a list of instructions exact enough for a machine to follow without guessing. Python carries them out one line at a time, from the top, doing exactly what you wrote.

KEY TAKEAWAY

Run small pieces often and look at what comes back. When Python stops, read the last line of the error first.

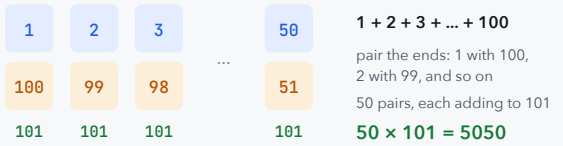
a task → exact steps → Python code in a .py file → python3 runs it line by line → output → or an error message

1 A program is a list of exact steps



A **program** is a list of instructions for a computer; the steps themselves are an **algorithm**. A program takes **input**, follows its steps and gives **output**. Steps must be exact: a person copes with "some paste", a computer needs an amount. A step can be a **decision**, which Python writes with `if` (see the conditionals sheet).

2 Four habits turn a task into steps



1 + 2 + 3 + … + 100

pair the ends: 1 with 100, 2 with 99, and so on

50 pairs, each adding to 101

50 × 101 = 5050

Decomposition: split a big job into parts (an essay into an outline). **Pattern recognition**: spot what repeats (after the first drawer, the others take the same steps). **Abstraction**: keep only the details that matter (an email account needs your name, not your favourite dish). **Algorithm**: write the steps down. Above, a pattern turns 99 additions into one multiplication.

3 `print` shows values; quotes make text

hello.py: what you write

```
1 # my first program
2 print("Hello, World!")
3 print("Python", 3, "is fun")
4 print()
5 print(7 + 5)
6 print("7 + 5")
```

output: what you see

```
nothing: a comment
Hello, World!
Python 3 is fun
(an empty line)
12
7 + 5
```

A **function** is a named command. You **call** it with round brackets around its **arguments**. Text in quotes is a **string**, printed as typed; without quotes Python works out the value. `print` puts one space between arguments. A line starting with `#` is a **comment**: a note for people, skipped by Python.

4 Python runs one line at a time

line	code	x	y	z	output
1	`print("Hello World!")`				`Hello World!`
2	`x = 10`	10			
3	`y = 10`	10	10		
4	`z = x + y`	10	10	20	
5	`print(z)`	10	10	20	20

blue: set on this line · grey: kept from before · blank: name not created yet

`python3 hello.py` runs a saved file. The **interpreter** (the program `python3`) does line 1, then line 2, and so on down. `x = 10` is an **assignment**: work out the right side, then attach the name on the left, a **variable**. A **trace table** like this predicts output by hand; the lecture's `hello.py` pauses between lines with a debugger.

5 A spreadsheet is a program too

C2 | *fx* `=IF(B2>90, "A", "B")` grade A if marks above 90

	A	B	C
1	Name	Marks	Grade
2	Alice	80.5	B
3	Bob	92.1	A
4	Carol	83.2	B

before

type 95 into B4

C4 updates itself

	A	B	C
1	Name	Marks	Grade
2	Alice	80.5	B
3	Bob	92.1	A
4	Carol	95	A

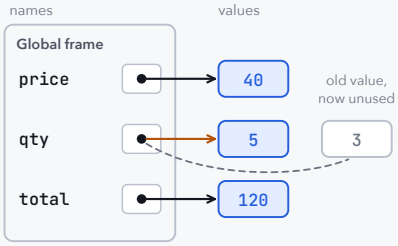
after: nobody retyped C4

A **cell** has an address (column, then row) and holds a value. A **formula** starts with = and reads other cells: C2 reads B2; copied down, C4 reads B4. The spreadsheet notebook's rule: above 90 is A, else B (so 90 gets B). Change an input and every formula that reads it recomputes.

6 Python runs a formula once

shop.py

```
1 price = 40
2 qty = 3
3 total = price * qty
4 print(total) → 120
5 qty = 5
6 print(total) → 120
```



green: what each print showed
amber: line 5 and the arrow it moved

A variable is like a cell, except nothing recomputes. Line 3 worked out 120 and attached `total` to it; line 5 points `qty` at 5 and leaves `total` alone. Repeat line 3 after line 5 and the next print gives 200. Names sit in a **frame**, arrows point to values, as on pythontutor.com.

Run it, then read what comes back

Two ways to run Python, the three errors you meet first, and how to read Python's error message.

7 Two ways to run Python: the REPL and a script

REPL: type python3, get >>>

```
>>> 2 + 3
5
>>> "hi"
'hi'
>>> print("hi")
hi
>>> x = 7
>>> x * 2
14
```

script: demo.py

```
2 + 3
"hi"
print("hi")
```

terminal

```
$ python3 demo.py
hi
```

the values of 2 + 3 and "hi" were computed, then thrown away

`python3` alone (or an online editor's console, such as Replit's) gives the >>> prompt of the **REPL**: read, evaluate, print, loop. It shows the value of every **expression** (code that has a value). A `.py` **script** shows only what you print. Try lines in the REPL; keep programs in scripts.

8 Errors come in three kinds

SyntaxError caught while reading the file: nothing runs

```
print("start")
print(Hello World!)
```

output *(nothing printed)*

```
SyntaxError: invalid syntax. Perhaps you forgot a comma?
```

NameError caught while running: line 1 already ran

```
print("start")
prnt("done")
```

output `start`

```
NameError: name 'prnt' is not defined. Did you mean: 'print'?
```

TypeError caught while running: line 1 already ran

```
print("start")
print("Seat " + 12)
```

output `start`

```
TypeError: can only concatenate str (not "int") to str
```

● ran ● Python stopped here ● never ran

A **SyntaxError** breaks Python's grammar and is found before anything runs. **Runtime errors** appear when Python reaches the line: a **NameError** is a name never created (a typo, a wrong capital, used too early); a **TypeError** is the wrong kind of value, as + cannot join text and a number. Fix: `"Seat " + str(12)`.

9 Read a traceback from the bottom up

```
Traceback (most recent call last):
  File ".../area.py", line 3, in <module>
    area = width * heigth
                   ^^^^^^
NameError: name 'heigth' is not defined. Did you mean: 'height'?
```

1. Last line: the kind of error, then what went wrong. Read it first.
2. File and line number: where Python stopped (folder cut to ...).
3. The line itself; the ^^^ marks point at the culprit.
4. Header: the chain of steps that led here (one step for now).

A runtime error prints a **traceback**. Here `area.py` misspells `height` on line 3; fixed, it prints 12. The "Did you mean" hint is a guess and is sometimes wrong.

10 What Python can build

Games	Desktop apps	Web apps	Databases
`turtle, pygame`	`tkinter`	`flask`	`sqlite3`
snake, 2048 and an 8-bit game	a Celsius and Fahrenheit window	a page, a form that saves records	a table of names, addresses, cities
Data analysis	**Maths**	**Generative art**	**Puzzles**
`pandas`	`sympy, jax`	`samila`	`plain Python`
count and plot a class list	limits and derivatives	pictures from random formulas	cipher: map → ocr Euler 1: 233168

Demos from the first lecture, with the **library** each loads with `import`. The puzzle answers come from running the lecture's code.

11 Common mistakes: what you wrote, what Python says, the fix

you wrote	the last line Python prints	why, and the fix
`print("Hi)`	`SyntaxError: unterminated string literal (detected at line 1)`	close the quote
`print("Hi"`	`SyntaxError: '(' was never closed`	add the missing)
`print(“Hi”)`	`SyntaxError: invalid character '“' (U+201C)`	type straight quotes
`Print("Hi")`	`NameError: name 'Print' is not defined. Did you mean: 'print'?`	names are case-sensitive
`  print("Hi")`	`IndentationError: unexpected indent`	delete the two leading spaces
`total = price * 3`	`NameError: name 'price' is not defined. Did you mean: 'print'?`	create `price` first

12 Try it (answers in green)

1. `print("3 + 4")`, then `print(3 + 4)`?
   `3 + 4`, then 7: quotes keep text.
2. `a = 2, b = a + 3, a = 10, print(a, b)`?
   `10 5`: `b` was set when `a` was 2.
3. Fix `print(Good morning)`.
   `print("Good", "morning")` shows `Good morning`.
4. `print("one")`, then `print("two"`. Is "one" shown?
   No: a SyntaxError, so nothing runs.
5. A file holds only `10 * 2`. Script output? REPL?
   Script: nothing. REPL: `20`.
6. Fix `print("Total: " + 5 * 4)`.
   `print("Total: " + str(5 * 4))` shows `Total: 20`.

exact steps · run top to bottom, once · print shows values · read errors from the last line up

Source: introductory lecture slides (computing around us, computational thinking), the demos in notebooks/Introduction and the spreadsheet notebook, nipunbatra.github.io/comp22. Every output and error message was produced by running the code with Python 3.11.11; other versions word some messages differently.